Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester(2025-26)
Faculty Name	Ms. Alluri Swathi
Student Name	N. Tejasri
Roll Number	2520030132
Branch	CSE
Course Outcome	1

Case Study Topic: SecureVote – Digital Voting & Election Analytics System using BST

Work Done Summary:

Implemented a **Binary Search Tree (BST)** in Java keyed by **voterId / voteTimestamp**. Inserted multiple voter records and vote entries into the BST. Implemented voter registration, secure ballot casting, vote counting, election analytics, fraud detection, and auditing operations. Deleted invalid or duplicate vote records and maintained proper BST structure. Implemented traversal methods for displaying election results in sorted order. Added methods for searching voters quickly and generating top election statistics. Compared BST performance with ArrayList – BST provides faster searching and organized data handling for election systems.

Implementation Details :

1. Created BST Node class with fields: key, voterId, candidateName, left, right.
2. insert( ) recursively inserts new voter/vote records into the BST.
3. search( ) finds voter details and validates authentication.
4. delete( ) removes duplicate/fraudulent vote records.
5. inorderTraversal( ) displays votes and election results in sorted order.
6. voteCount( ) calculates total votes received by each candidate.
7. fraudDetection( ) checks duplicate voting attempts.
8. Main method demonstrated voter registration, vote casting, searching, deletion, analytics, and result reporting.

Result : Screenshots/Output

```
PS D:\JavaPrograms> javac SecureVoteBST.java
PS D:\JavaPrograms> java SecureVoteBST
BST Inorder Traversal:
Voter ID: 20 -> Candidate: Candidate A
Voter ID: 30 -> Candidate: Candidate B
Voter ID: 40 -> Candidate: Candidate B
Voter ID: 50 -> Candidate: Candidate A
Voter ID: 60 -> Candidate: Candidate C
Voter ID: 70 -> Candidate: Candidate C
Voter ID: 80 -> Candidate: Candidate A

Voter ID 40 Found

Deleting Voter ID 20...

BST After Deletion:
Voter ID: 30 -> Candidate: Candidate B
Voter ID: 40 -> Candidate: Candidate B
Voter ID: 50 -> Candidate: Candidate A
Voter ID: 60 -> Candidate: Candidate C
Voter ID: 70 -> Candidate: Candidate C
Voter ID: 80 -> Candidate: Candidate A

Election Results:
Candidate C = 2 votes
Candidate B = 2 votes
Candidate A = 2 votes
```

Time Complexity:

- BST Insert/Delete/Search: **O(log n)** (average case)
- Vote Counting Traversal: **O(n)**
- Compared to ArrayList searching **O(n)**, BST performs better for large-scale election data management.

GitHub Link : https://github.com/tejasri3123/Dsa-2

Student Signature	**Faculty Signature**